

Pawnbroking

New Shop Installer & Onboarding Guide

One-step setup for a fresh Windows machine, cloud provisioning (single + multi-shop), sync agent service, and Andri

Bundle: ShopInstallerBundle\

Includes multi-shop access & Switch Shop menu

Contents

Welcome

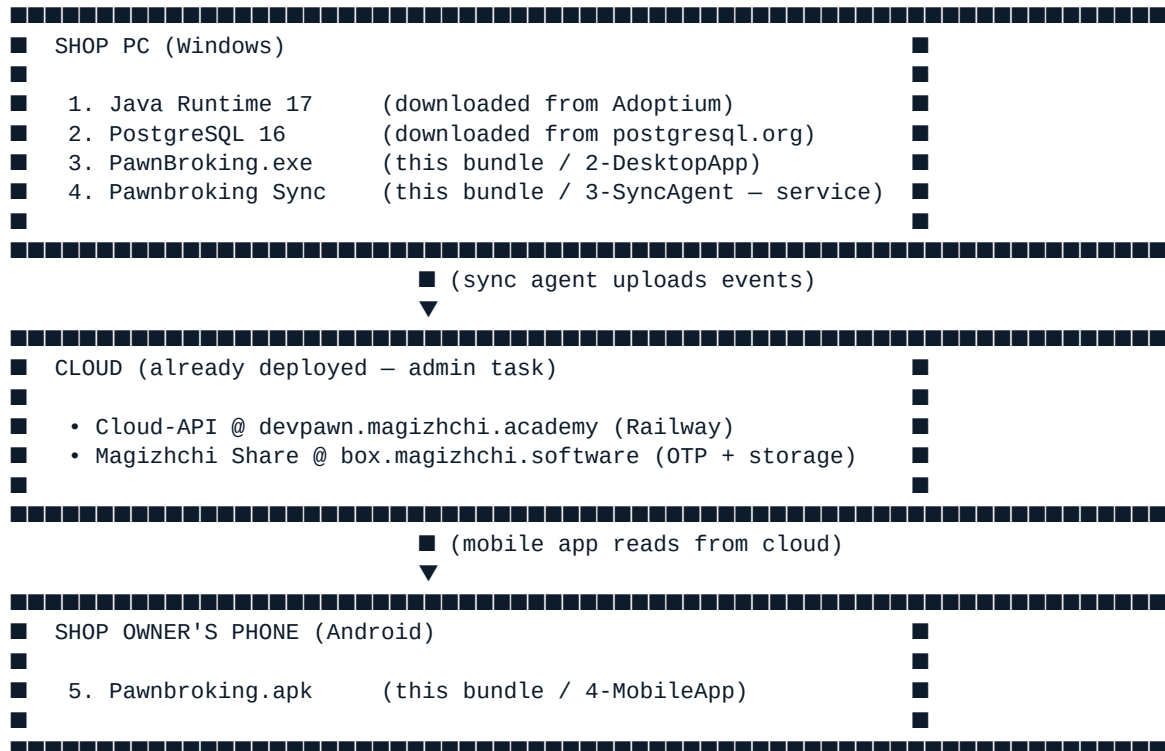
1. Step 0 — Admin / Cloud Provisioning
2. Step 1 — Prerequisites
3. Step 2 — Desktop App
4. Step 3 — Sync Agent
5. Step 4 — Mobile App
6. Step 5 — One-time SQL

Welcome

This bundle contains EVERYTHING needed to set up a new shop on a new computer + connect their phone. Follow the folders in order — each has its own README.txt with detailed instructions.

For a screenshot-rich walkthrough, see `New_Shop_Onboarding.pdf` (in this same folder).

AT A GLANCE — what gets installed where



STEP-BY-STEP — do these in order

STEP 0 — Admin / Cloud Provisioning (you, before visiting shop)

```
Open 0-AdminProvisioning\README.txt
Add shop_id to Railway TENANTS env var, insert tenant +
primary_email rows, generate the unique API key for this
shop, paste it into the bundle's sync.properties.
Without this, Step 3 will fail with HTTP 401.
```

STEP 1 — Prerequisites

```
Open 1-Prerequisites\README.txt
Install Java 17 + PostgreSQL 16. Verify both work.
```

STEP 2 — Desktop App

```
Open 2-DesktopApp\README.txt
Run PawnBrokingSetup.exe. First-launch wizard creates the DB
schema. Create the shop's first company (CMP1).
```

STEP 3 — Sync Agent

Open 3-SyncAgent\README.txt
Copy folder to C:\Program Files\PawnbrokingSync\
fill in sync.properties (db.password + the rest is already
set from Step 0), run install-service.bat as ADMIN.

■■ Step 0 must be done BEFORE this step ■■
Otherwise the agent will run but events will be rejected
with "tenant not configured" or "bad api key".

STEP 4 — Mobile App

Open 4-MobileApp\README.txt
Send Pawnbroking.apk to the shop owner's phone. Install.
Login with the gmail address added in Step 3.

STEP 5 — One-time SQL (only if something looks empty)

Open 5-OneTimeSQL\README.txt
Run these scripts only if the mobile app shows ■0 for
sections that clearly have data on the desktop.

TYPICAL TIMINGS

Step 0	(admin / cloud prov)	..	5 min	(done before visiting)
Step 1	(prerequisites)	..	15-30 min	(PostgreSQL is slow)
Step 2	(desktop)	..	5 min	
Step 3	(sync agent)	..	5 min	
Step 4	(mobile app)	..	3 min	
Step 5	(replays, if needed)	..	2 min	
				■■■■■■■■■■
Total			~35-50 min	for a brand-new shop

WHAT TO PROVIDE THE SHOP BEFORE YOU LEAVE

- ✓ Their shop_id (e.g. "balamurugan")
- ✓ The gmail address (used for mobile OTP login)
- ✓ The PostgreSQL password (in case they need to reinstall)
- ✓ The cloud API key (already in sync.properties)
- ✓ Phone number / email for support

TROUBLESHOOTING QUICK REFERENCE

Desktop app won't open

- Check Java is installed: `java -version`
- Check PostgreSQL is running: Start menu → "Services" → find "postgresql-x64-16" → should be "Running".

Sync Agent service won't start

- Check `pawnbroking-sync.err.log` in the sync agent folder.
- Most common cause: wrong `db.password` in `sync.properties`.

Mobile app shows "Too many OTP requests"

- Wait 60 seconds. The box rate-limits OTP send to one per minute per email.

Mobile app shows "tenant not configured"

- Admin hasn't added the `shop_id` to the cloud's `TENANTS` env var. See Step 3 admin action.

Mobile app shows ■0 / blank for a section that has data

- Run the relevant replay SQL from Step 5.

Mobile app shows old data, missing latest bills

- Sync agent isn't running. Open `services.msc` → start "Pawnbroking Sync Agent".

Step 0 — Admin / Cloud Provisioning

WHO RUNS THIS

Only the admin / integrator (you). NOT the shop owner.
The shop owner never sees this step – they just receive their API key inside a pre-configured sync.properties file.

WHEN TO RUN

Before Step 3 (Sync Agent). Without this, the agent will run but every event it sends will be rejected with HTTP 401 "bad api key" or 404 "tenant not configured".

WHAT THIS DOES

1. Registers the new shop_id in the cloud database.
2. Adds the shop owner's gmail to the email allowlist (needed for mobile OTP login).
3. Generates a unique API key that the sync agent will use to authenticate every event upload.

PART A — Add shop to Railway env var

1. Open <https://railway.app> → log into your account.
2. Open the "pawnbroking-cloud-api" project → Variables tab.
3. Find the variable: TENANTS
4. Append the new shop_id (comma-separated), e.g.:

```
before: alwarpuram,annanagar,mylocal
after : alwarpuram,annanagar,mylocal,balamurugan
```

5. Save → Railway will auto-redeploy (~3 minutes).
6. Verify in the deployment logs:

```
"Provisioning tenant schema 'balamurugan'..."
"Flyway migrations applied to balamurugan: 3"
```

The tenant schema (balamurugan.projections, balamurugan.notifications, etc.) is created automatically by Flyway on cloud startup.

PART B — Provision shop + generate API key (one SQL block)

Connect to the cloud Postgres in Railway (Data tab → Query).

Edit the three values at the top, paste the rest as-is, then Execute:

```
DO $$
DECLARE
    v_shop_id TEXT := 'balamurugan';
    v_email    TEXT := 'balamurugan.pawn@gmail.com';
    v_label    TEXT := 'Balamurugan shop - sync agent';
    v_api_key  TEXT;
BEGIN
    v_api_key := 'mbk_' || translate(
        encode(gen_random_bytes(24), 'base64'),
        '+/=', 'xyz');

    INSERT INTO public.tenants (shop_id)
    VALUES (v_shop_id)
    ON CONFLICT DO NOTHING;

    INSERT INTO public.tenant_primary_email (shop_id, email)
    VALUES (v_shop_id, v_email)
```

```

ON CONFLICT DO NOTHING;

INSERT INTO public.shop_credentials (api_key, shop_id, label)
VALUES (v_api_key, v_shop_id, v_label);

RAISE NOTICE '=====';
RAISE NOTICE '  shop_id  : %', v_shop_id;
RAISE NOTICE '  email    : %', v_email;
RAISE NOTICE '  API KEY   : %', v_api_key;
RAISE NOTICE '=====';
END $$;

```

Railway shows the NOTICE output in the query result panel — copy the API key from there. It will look like:

```
mbk_xK8pQ7vR2sNzL4tBwHfYqMjE9aCdGoUixyz
```

PART C — Write the API key into sync.properties

Edit the file that will be shipped with the installer bundle:

```
3-SyncAgent\sync.properties      (NOT the .sample – your real one)
```

Set these three lines:

```
shop.id=balamurugan
cloud.api_key=mbk_xK8pQ7vR2sNzL4tBwHfYqMjE9aCdGoUixyz
db.password=<the PostgreSQL password the shop owner will choose>

```

The db.password will be filled in on-site during Step 1 (the shop owner picks it during PostgreSQL install). Leave it blank in your master copy or use a placeholder.

REVOKING A KEY (if compromised / shop owner leaves)

```

UPDATE public.shop_credentials
  SET revoked_at = now()
 WHERE api_key = 'mbk_xK8pQ7vR2sNzL4tBwHfYqMjE9aCdGoUixyz';

```

The next request from that key returns HTTP 401. Generate a fresh key with the SQL block above and update the shop's sync.properties.

AUDIT — see who has keys

```

SELECT shop_id, label, created_at, revoked_at,
       LEFT(api_key, 10) || '...' AS key_preview
  FROM public.shop_credentials
 ORDER BY created_at DESC;

```

Only the first 10 chars of each key are shown — the full key was already given to the shop owner and is never recoverable from here (you'd just generate a new one and revoke the old).

TROUBLESHOOTING

Agent log shows "HTTP 401 bad api key"

```
→ Key in sync.properties doesn't match any row in
   public.shop_credentials, OR the row's revoked_at is non-NULL.
```

Agent log shows "tenant not configured"

```
→ shop_id is not in Railway's TENANTS env var, OR the cloud
   hasn't been restarted since you updated the env var.
```

Mobile app shows "email not allowed"

```
→ public.tenant_primary_email is missing the row, OR the email
   in the app does not match exactly (case-sensitive). Fix:
```

```
SELECT * FROM public.tenant_primary_email WHERE shop_id = 'balamurugan';
```

CHECKLIST — done with Step 0 when:

- [] shop_id appears in Railway TENANTS env var
- [] Railway logs show "Provisioning tenant schema '<shop_id>'"
- [] public.tenants has the new row
- [] public.tenant_primary_email has the gmail row
- [] public.shop_credentials has a fresh non-revoked api_key row
- [] public.user_shop_access has the (email, shop_id) row
- [] API key copied into the bundle's 3-SyncAgent\sync.properties
- [] shop_id and email written down for handoff to shop owner

After all seven are checked, you're ready to visit the shop and run Steps 1-4 on their machines.

MULTI-SHOP ACCESS — give one user access to several shops

WHAT THIS IS

If a shop owner runs more than one shop (e.g. a chain), they can sign into the mobile app once and switch between shops via the Home screen → ■ overflow → "Switch Shop" menu.

HOW IT WORKS

public.user_shop_access maps an email to one or more shop_ids. On OTP login the cloud collects every shop the email has access to:

- exactly 1 shop → app goes straight to Home for that shop
- 2+ shops → app shows a picker; user taps the shop they want to open

GRANT ACCESS TO AN ADDITIONAL SHOP

Connect to the cloud Postgres (Railway → Data tab) and run:

```
INSERT INTO public.user_shop_access (email, shop_id, role)
VALUES ('owner@example.com', 'balamurugan', 'OWNER')
ON CONFLICT (email, shop_id) DO NOTHING;
```

No restart, no APK reinstall. On the user's next login (or next Switch Shop tap) the new shop appears in the picker.

Roles available: OWNER (default) / VIEWER / AUDITOR. All three have the same read access today; differentiation can be added later if needed.

REVOKE ACCESS WITHOUT DELETING THE AUDIT TRAIL

```
UPDATE public.user_shop_access
SET revoked_at = now()
WHERE email = 'owner@example.com' AND shop_id = 'balamurugan';
```

The next request from that user for that shop returns HTTP 403. Reactivate later by clearing revoked_at to NULL.

AUDIT — see who has access to what

```
SELECT email, shop_id, role, added_at, revoked_at
FROM public.user_shop_access
ORDER BY email, shop_id;
```

TROUBLESHOOTING — multi-shop edge cases

Picker shows fewer shops than expected

1. Confirm the row exists:

```
SELECT * FROM public.user_shop_access
WHERE lower(email) = 'owner@example.com';
```

2. Confirm both tenants are active:
SELECT shop_id, active FROM public.tenants;
3. Confirm no revoked_at:
SELECT * FROM public.user_shop_access
WHERE email = 'owner@example.com' AND revoked_at IS NOT NULL;

Picker shows "Forbidden" when tapping a shop

Cloud-api is older than the multi-shop /select-shop fix. Verify
Railway is on the latest deploy.

User can't tap Switch Shop at all

They're using an older APK without the menu item. Reinstall from
4-MobileApp\Pawnbroking.apk in this bundle.

Step 1 — Prerequisites

These two free programs MUST be installed first. Anything below will fail to start if they are missing.

A. Java Runtime 17 (for the desktop app + sync service)

Download:

```
https://adoptium.net/temurin/releases/?version=17&os=windows
```

Pick: Windows · x64 · JDK 17 · .msi installer

Run the .msi → accept defaults → tick "Add to PATH" if asked.

Verify (open Command Prompt):

```
java -version
```

Expected output: "openjdk version "17.0.x"..."

B. PostgreSQL 16 (the local database)

Download:

```
https://www.postgresql.org/download/windows/  
→ click "Download the installer" (EDB) → pick PostgreSQL 16 · Windows x64.
```

During the install wizard:

- Components: keep everything ticked (Server + pgAdmin 4 + Stack Builder)
- Data directory: leave default
- PASSWORD for the "postgres" user: PICK ONE AND WRITE IT DOWN
(you will need it in step 3)
- Port: 5432 (leave default)
- Locale: Default
- Skip Stack Builder at the end

After install, open Start menu → "pgAdmin 4" → connect with the password you just set. You should see a database tree on the left.

C. (recommended) Modern Chrome / Edge browser for pgAdmin

pgAdmin opens in your default browser. Make sure it is up to date.

Once both Java AND PostgreSQL are installed and verified, move on to step 2 (Desktop App).

Step 2 — Desktop App

WHAT THIS IS

The Windows program the shop staff uses every day to open / close bills, take advances, record expenses, etc. It writes to the local PostgreSQL database installed in Step 1.

INSTALLATION

1. Double-click PawnBrokingSetup.exe (in this folder).
2. Accept defaults. The app will be installed to:
C:\Program Files\PawnBroking\
and a shortcut placed on the Desktop / Start menu.
3. Launch "PawnBroking" from the Desktop shortcut.
4. First-run wizard will ask for the PostgreSQL password you set in Step 1.B – enter it. The app will then create all required tables and the very first company (CMP1).
5. Configure the shop:
 - Company name (e.g. "Balamurugan Pawn Broking")
 - Address / phone number
 - Backup folder (any local path with plenty of space)
 - Image folder for bill photos

DAILY USE

The desktop app works completely offline – it never needs internet. The sync agent (Step 3) handles all the cloud uploading in the background. If your internet drops, nothing on the desktop breaks.

VERIFY

Open the app → create one test customer → create one test bill → close the app cleanly. Data should persist on next open.

When the desktop app is working, move on to Step 3 (Sync Agent).

Step 3 — Sync Agent

WHAT THIS IS

A small Windows service that runs silently in the background. It watches the local PostgreSQL database for changes and ships them to the Magizhchi cloud so the mobile app can see live data. It also uploads bill images and the daily backup folder to Magizhchi Share.

WHAT IS IN THIS FOLDER

pawnbroking-sync-agent.jar	← the actual program
pawnbroking-sync.exe	← Windows service wrapper (winsw)
pawnbroking-sync.xml	← service config (don't edit)
sync.properties.sample	← config template – DO NOT install this as-is. Copy it first (see step 1 below).
install-service.bat	← run AS ADMIN after configuring

INSTALLATION (do these in order)

1. Copy this WHOLE folder to a permanent location, e.g.

```
C:\Program Files\PawnbrokingSync\
```

2. Make a copy of sync.properties.sample called sync.properties (drop

the ".sample"). Open it in Notepad and fill in:

```
db.password=<the PostgreSQL password you set in Step 1.B>
shop.id=<your shop_id – assign one, e.g. "balamurugan">
cloud.api_key=<we will give you this per-shop key>
```

Leave the rest (cloud.url, batch.size, etc.) untouched unless instructed otherwise.

3. ASK CLAUDE / THE ADMIN TO PROVISION THIS SHOP IN THE CLOUD:

- Add shop_id to the cloud's TENANTS env var on Railway
- Add a primary_email row in the public.tenant_primary_email table for the shop owner's gmail address
- Restart the cloud-api once so Flyway provisions the new tenant schema

Without this step, the sync agent will run but its events will be rejected by the cloud with "tenant not configured".

4. RIGHT-CLICK install-service.bat → Run as administrator.

It will register "pawnbroking-sync" as a Windows service that starts automatically with Windows.

5. Verify the service is running:

```
Open services.msc → look for "Pawnbroking Sync Agent" → status
should be "Running". OR run from the folder:
pawnbroking-sync.exe status
```

The first time it starts it will create the sync_outbox table and triggers in your DB automatically (SchemaGuard). No manual psql.

LOGS (if anything goes wrong, check these first)

pawnbroking-sync.wrapper.log	← service lifecycle (start/stop)
pawnbroking-sync.out.log	← normal output (every event sent)
pawnbroking-sync.err.log	← errors (cloud rejections, DB issues)

DAILY BEHAVIOUR

- Polls the local DB every 5 seconds for new events
- Batches 200 events at a time → POSTs to cloud
- Watches the bill-image folder once per minute → uploads new files
- Watches the backup folder → uploads new backup files
- All throttled to respect the Magizhchi box's 60 requests/minute

If internet is down, events accumulate locally and ship as soon as the connection returns. Nothing is lost.

MAINTENANCE COMMANDS (from this folder, in a Command Prompt)

<code>pawnbroking-sync.exe status</code>	← is it running?
<code>pawnbroking-sync.exe stop</code>	← stop the service
<code>pawnbroking-sync.exe start</code>	← start the service
<code>pawnbroking-sync.exe uninstall</code>	← remove the service entirely

Step 4 — Mobile App

WHAT THIS IS

The Android app the shop owner uses to view stock, today's account, monthly report, and bill images from anywhere – via the cloud.

WHAT IS IN THIS FOLDER

Pawnbroking.apk ← the Android installer

INSTALLATION

1. Transfer Pawnbroking.apk to the phone:

- Email it to yourself, OR
- Copy via USB cable, OR
- Upload to Google Drive and download on the phone

2. Tap the APK on the phone.

3. Android will warn "Install from unknown sources".

- Settings → tap "Allow from this source" → return → Install.

4. After install, the launcher will show an app named "Pawnbroking"

with the Pawnbroking logo.

FIRST LOGIN (passwordless OTP)

1. Open the app → enter the SAME gmail address that the admin added

to public.tenant_primary_email for this shop.
(e.g. balamurugan.pawn@gmail.com)

2. Tap "Send OTP". A 6-digit code is emailed to that gmail account

via Magizhchi Share.

- If you see "Too many OTP requests" – wait 60 seconds.

3. Open the email → copy the 6-digit code → paste in the app →

tap "Verify & Sign In".

4. You'll be signed in for 90 days (no need to login daily).

WHAT YOU CAN DO ON THE PHONE

- Home → monthly charts (count + amount) per company
- Stock Details → live opened/locked/repledged bills with filters
- Today's Account → 10-row operations grid matching desktop exactly
- MIS Report → last 12 months: bills opened, redeemed, profit
- Bell icon → real-time notifications for new bills, closings
- Bill detail → full bill info + bill images stored in cloud

The toolbar shows just the bell on the right. Tap the 3-dot menu (■) on the far right for:

- Switch Shop → only appears when you have access to 2+ shops
- Settings
- Logout

MULTI-SHOP USAGE (if your account has access to >1 shop)

The very first time you sign in:

1. Enter email + OTP as usual.
2. After Verify, a "Select a Shop" screen appears with one card per shop you have access to.
3. Tap a card → land on Home for that shop.

Later, to switch:

1. Home → tap the 3-dot menu (■) → Switch Shop.
2. Same picker reappears → tap a different card → Home reloads with the chosen shop's data.

The active shop is shown in the Home toolbar title. The bell badge, charts, Today's Account, and MIS Report all reflect the active shop.

The mobile app is READ-ONLY. All data entry still happens on the desktop. The mobile app reflects whatever the desktop wrote within a few seconds (as long as the Sync Agent is running).

Step 5 — One-time SQL

WHAT THIS IS

When a shop is migrated to the new system, some existing tables already have data – but the sync_capture trigger was added later, so those rows never generated sync events. These SQL scripts force every existing row to re-fire the trigger, shipping them to the cloud in one go.

WHEN TO RUN THEM

ONLY run a script after:

1. The Sync Agent (Step 3) is installed and running.
2. The shop has been provisioned in the cloud (TENANTS env var).
3. You see ZEROS in the mobile app for that section even though the desktop clearly has data.

HOW TO RUN

1. Open pgAdmin → connect to the local PostgreSQL.
2. Open the relevant .sql file from this folder.
3. EDIT the line ``SET LOCAL app.shop_id = 'mylocal';`` to your shop's actual shop_id (e.g. 'balamurugan').
4. Click "Execute" (F5).
5. Wait ~30 seconds – the sync agent will flush the new events to the cloud. Reopen the mobile app screen to verify.

WHAT EACH SCRIPT DOES

replay_todays_account.sql

Backfills: company_todays_account
 company_todays_account_available_amount
 company_advance_amount

Run this if Today's Account shows ■0 for everything despite desktop having day-close data.

replay_other_debit_credit.sql

Backfills: company_other_debit (Expenses)
 company_other_credit (Incomes)

Run this if the EXPENSES / INCOMES rows on Today's Account show 0 / 0 even though desktop has expense entries.

replay_expense_income.sql

Generic auto-detect script. Finds any table whose name contains "expense", "income", "entry", or "voucher" and replays it. Useful if you've added custom tables we don't know about.

SAFETY

- These scripts do NOT change any data – they only fire the sync trigger via no-op UPDATES.
- Safe to re-run any number of times.
- Wrapped in a transaction; if anything fails, nothing is committed.